

Byzantine Agreement

May 14, 2026

0.1 Accordo Bizantino - Analisi di un esempio pratico:

Quattro amici vogliono andare assieme al cinema e sono indecisi tra due film. Il primo film e' in centro, il secondo in periferia. Tre dei quattro amici sono sinceramente interessati a trovare un accordo, mentre il quarto e' dispettoso. In caso di pareggio la scelta ricade sul cinema in centro. Se due amici vogliono andare in periferia e il terzo in centro, mostrare come la combinazione dei messaggi dell'amico dispettoso e gli esiti del lancio della moneta globale possono portare al raggiungimento dell'accordo sia sul cinema in centro sia sul cinema in periferia.

Analizziamo il problema:

Questo e' un classico problema decisionale fra n processi, di cui f *faulty*, in cui ogni processo ha un valore in input x_i e un output y_i .

L'algoritmo proposto per risolvere il Problema del Consensus deve soddisfare i seguenti requisiti:

- 1) **Agreement:** I processi affidabili vogliono accordarsi su uno stesso output Y scelto fra i possibili X input
- 2) **Validity:** Se tutti i processi affidabili hanno lo stesso input X , allora devono sceglierlo come output
- 3) **Termination:** Tutti i processi devono terminare dopo un numero finito di passi

Per il vincolo di **Validita'** il raggiungimento del consenso non puo' essere basato su un valore concordato di *default*.

Il consenso puo' essere raggiunto se solo se: $n \geq 3f + 1$.

Una prima soluzione a questo problema e' proposta sotto forma dell'algoritmo **Deterministico Exponential Information Gathering** o EIG che consente il raggiungimento di un accordo in $f+1$ *round*, curioso notare l'analogia fra un processo 'dispettoso' e uno che fallisce.

Il protocollo prevede che, per tutti i *round*, ogni processo affidabile p_j spedisca **sempre** il valore iniziale $b_0(j)$ e costruisca un albero che cresce di un livello per *round*. Senza entrare nei dettagli del suo funzionamento, l'algoritmo ha complessita' **lineare**, mentre il costo dei messaggi cresce **esponenzialmente** col numero dei *round* (per questo *Exponential*).

Proponiamo adesso un algoritmo molto piu' efficiente, l'algoritmo **randomizzato** di tipo *Monte Carlo*. Per far si che funzioni, si aggiunge un'ulteriore ipotesi, ad ogni *round* e' comunicato a tutti gli n processi il risultato del lancio di una **moneta globale onesta** (con probabilita' $\mathbb{P}(\text{testa}) = \mathbb{P}(\text{croce}) = \frac{1}{2}$), si potrebbe pensare che si tratti di un *valore di default* sul quale potersi accordare, ricordiamo pero' il vincolo di **Validity** e dunque non e' questo il caso.

Introduciamo infine la soglia $T = 2f + 1$ ($n \geq 3f + 1 \Rightarrow n - f \geq 2f + 1$) che rappresenta il numero di processi necessari affinché si raggiunga un consenso.

MCByzantineAgreement, sia $b(j) \in \{0, 1\}$ il valore del *bit* del processo p_j per $j = 1, \dots, n - f$, prende in **Input** $b(j) = b_0(j)$ e **restituisce** con probabilit  almeno $\frac{1}{2}$, ad ogni round $b(j) = v$.

MCByzantineAgreement($b(j)$)

while(TRUE)

1. trasmetti $b(j)$ agli altri $n-1$ processi
2. ricevi i valori spediti dagli altri $n-1$ processi
3. $\text{maj}(j) \leftarrow$ valore maggioritario tra i ricevuti (incluso il proprio)
4. $\text{tally}(j) \leftarrow$ numero dei valori uguali a $\text{maj}(j)$
5. if $\text{tally}(j) \geq T$
 - then $b(j) \leftarrow \text{maj}(j)$
 - else if testa
 - then $b(j) \leftarrow 1$
 - else $b(j) \leftarrow 0$

Avremo quindi che:

- 1) Durante ogni *round* ognuno degli n processi spedisce un messaggio a ogni altro processo,
- 2) Prima dell'inizio di un nuovo *round* ogni processo ha ricevuto un messaggio da ogni altro processo e infine che,
- 3) Nello stesso *round* ogni processo affidabile spedisce il proprio messaggio a ogni altro processo senza alterarlo.

Sono possibili due casi:

- 1) I $n - f$ processi affidabili sono **unanimente d'accordo** e dunque il consenso   raggiunto. Questo poich  al passo 5, per un qualunque processo affidabile j , $\text{tally}(j) \geq T$ in un qualunque *round*.
- 2) **Non tutti i processi affidabili concordano**, qui si considerano due casi.
 - 2.1) Se non vi   alcun processo affidabile per cui $\text{tally}(j) \geq T$, ossia non si ha un numero di valor maggioritari superiore alla soglia, tutti i processi seguiranno il lancio della moneta globale e dunque il consenso si trover  al *round successivo*.
 - 2.2) In caso contrario, quindi se esiste un processo j^* per il quale $\text{tally}(j^*) \geq T$, poich  i processi *faulty* sono al piu' f e per almeno $f + 1$ processi affidabili il bit spedito   $\text{maj}(j^*)$ ($T - f = f + 1$), inoltre, nello stesso *round* non puo' esserci alcun processo affidabile k^* che superi l'if con valor $\text{maj}(k^*) \neq \text{maj}(j^*)$ perche' non ci sono altrettanti processi votanti e viene inviato un solo messaggio per round. Pertanto con probabilit  $\frac{1}{2}$, dopo l'esito del lancio della moneta, tutti i processi affidabili al *round* successivo convergono su $\text{maj}(j^*)$.

Questo algoritmo converge in tempo **costante** e il costo delle comunicazioni   **lineare** al numero di *round*.

Poniamo dunque $n = 4$ e $f = 1$, in questo caso, il consenso puo' essere raggiunto in quanto: $n \geq 3f + 1$, infatti $4 = 4$.

Assegnamo al **cinema in centro** il bit 0, mentre al **cinema in periferia** il bit 1 e denominiamo i quattro processi in ordine alfabetico (A, B, C, D)

Come rappresentazione scelgo la Tupla: <id processo, valore processo, valori ricevuti, valor maggioritario, tally>.

Analizziamo il primo caso in cui l'accordo ricade sul cinema in centro, CROCE, per il processo A:

```
// — ROUND 0 — //
1 <A, 1, x, x, x, x, x> // trasmetti b(j) agli altri n-1 processi
2 <A, 1, 1, 0, 0, x, x> // ricevi i valori spediti dagli altri n-1 processi
3 <A, 1, 1, 0, 0, 0, x> // maj(j) <- valore maggioritario tra i ricevuti
4 <A, 1, 1, 0, 0, 0, 2> // tally(j) <- numero dei valori uguali a maj(j)
5 <A, 0, 1, 0, 0, 0, 2> // tally(j) < T e Croce - SYNC
// — ROUND 1 — //
1 <A, 0, x, x, x, x, x> // trasmetti b(j) agli altri n-1 processi
2 <A, 0, 0, 0, 0, x, x> // ricevi i valori spediti dagli altri n-1 processi
3 <A, 0, 0, 0, 0, 0, x> // maj(j) <- valore maggioritario tra i ricevuti
4 <A, 0, 0, 0, 0, 0, 4> // tally(j) <- numero dei valori uguali a maj(j)
5 <A, 0, 0, 0, 0, 0, 4> // Accordo
```

In questo caso l'amico dispettoso **D** ha creato una situazione di pareggio per il processo **A**, dunque si e' ricaduti sul valore 0 di *default* che e' stato confermato dal lancio della moneta globale *Croce*. quindi al *round* successivo il consenso e' raggiunto in quanto tutti e tre i processi affidabili si sono sincronizzati sul valore della moneta (supponendo che **B**, **C** si comportino analogamente).

Analizziamo il secondo caso in cui l'accordo ricade sul cinema in periferia, TESTA, sempre per il processo A:

```
// — ROUND 0 — //
1 <A, 1, x, x, x, x, x> // trasmetti b(j) agli altri n-1 processi
2 <A, 1, 1, 0, 0, x, x> // ricevi i valori spediti dagli altri n-1 processi
3 <A, 1, 1, 0, 0, 0, x> // maj(j) <- valore maggioritario tra i ricevuti
4 <A, 1, 1, 0, 0, 0, 2> // tally(j) <- numero dei valori uguali a maj(j)
5 <A, 1, 1, 0, 0, 0, 2> // tally(j) < T e Testa - SYNC
// — ROUND 1 — //
1 <A, 1, x, x, x, x, x> // trasmetti b(j) agli altri n-1 processi
2 <A, 1, 1, 1, 0, x, x> // ricevi i valori spediti dagli altri n-1 processi
3 <A, 1, 1, 1, 0, 1, x> // maj(j) <- valore maggioritario tra i ricevuti
4 <A, 1, 1, 1, 0, 1, 3> // tally(j) <- numero dei valori uguali a maj(j)
5 <A, 1, 1, 1, 0, 1, 3> // Accordo
```

Simile, il caso in cui esce *Testa*, i processi affidabili inizialmente in disaccordo a causa dell'operato di **D**, trovano il consenso al secondo *round*.

E' facile notare come i messaggi dell'amico dispettoso in questo caso siano stati completamente ignorati. Vediamo ora invece il seguente esempio in cui uno dei processi affidabili raggiunge $tally(j) \geq T$ (caso 2.2):

Analizziamo il primo caso in cui l'accordo ricade sul cinema in periferia, TESTA:

```
// — ROUND 0 — //
1 <A, 1, x, x, x, x, x>, <B, 1, x, x, x, x, x>, <C, 0, x, x, x, x, x>
2 <A, 1, 1, 0, 0, x, x>, <B, 1, 1, 0, 0, x, x>, <C, 0, 1, 1, 1, x, x>
3 <A, 1, 1, 0, 0, 0, x>, <B, 1, 1, 0, 0, 0, x>, <C, 0, 1, 1, 1, 1, x>
4 <A, 1, 1, 0, 0, 0, 2>, <B, 1, 1, 0, 0, 0, 2>, <C, 0, 1, 1, 1, 1, 3>
5 <A, 1, 1, 0, 0, 0, 2>, <B, 1, 1, 0, 0, 0, 2>, <C, 1, 1, 1, 1, 1, 3> // TESTA
// — ROUND 1 — //
1 <A, 1, x, x, x, x, x>, <B, 1, x, x, x, x, x>, <C, 1, x, x, x, x, x>
2 <A, 1, 1, 1, 0, x, x>, <B, 1, 1, 1, 0, x, x>, <C, 1, 1, 1, 1, x, x>
3 <A, 1, 1, 1, 0, 1, x>, <B, 1, 1, 1, 0, 1, x>, <C, 1, 1, 1, 1, 1, x>
4 <A, 1, 1, 1, 0, 1, 3>, <B, 1, 1, 1, 0, 1, 3>, <C, 1, 1, 1, 1, 1, 4>
5 <A, 1, 1, 1, 0, 1, 3>, <B, 1, 1, 1, 0, 1, 3>, <C, 1, 1, 1, 1, 1, 4> // Accordo
```

In questo caso il processo **C** inizialmente, poiche' riceve 1 da **D**, trova un valore *maggioritario*, che viene propagato dal lancio della moneta casuale *Testa* agli altri due processi affidabili. Dunque si trova un accordo al *round* successivo.

Invece se **CROCE** scelgo il cinema in centro:

```
// — ROUND 0 — //
1 <A, 1, x, x, x, x, x>, <B, 1, x, x, x, x, x>, <C, 0, x, x, x, x, x>
2 <A, 1, 1, 0, 0, x, x>, <B, 1, 1, 0, 0, x, x>, <C, 0, 1, 1, 1, x, x>
3 <A, 1, 1, 0, 0, 0, x>, <B, 1, 1, 0, 0, 0, x>, <C, 0, 1, 1, 1, 1, x>
4 <A, 1, 1, 0, 0, 0, 2>, <B, 1, 1, 0, 0, 0, 2>, <C, 0, 1, 1, 1, 1, 3>
5 <A, 0, 1, 0, 0, 0, 2>, <B, 0, 1, 0, 0, 0, 2>, <C, 1, 1, 1, 1, 1, 3> // CROCE
// — ROUND 1 — //
1 <A, 0, x, x, x, x, x>, <B, 0, x, x, x, x, x>, <C, 1, x, x, x, x, x>
2 <A, 0, 0, 1, 0, x, x>, <B, 0, 0, 1, 0, x, x>, <C, 1, 0, 0, 1, x, x>
3 <A, 0, 0, 1, 0, 0, x>, <B, 0, 0, 1, 0, 0, x>, <C, 1, 0, 0, 1, 0, x>
4 <A, 0, 0, 1, 0, 0, 3>, <B, 0, 0, 1, 0, 0, 3>, <C, 1, 0, 0, 1, 0, 2>
5 <A, 0, 0, 1, 0, 0, 3>, <B, 0, 0, 1, 0, 0, 3>, <C, 0, 0, 0, 1, 0, 2> // CROCE - SYNC
// — ROUND 2 — //
1 <A, 0, x, x, x, x, x>, <B, 0, x, x, x, x, x>, <C, 0, x, x, x, x, x> //
2 <A, 0, 0, 1, 0, x, x>, <B, 0, 0, 1, 0, x, x>, <C, 0, 0, 0, 1, x, x> //
3 <A, 0, 0, 1, 0, 0, x>, <B, 0, 0, 1, 0, 0, x>, <C, 0, 0, 0, 1, 0, x> //
4 <A, 0, 0, 1, 0, 0, 3>, <B, 0, 0, 1, 0, 0, 3>, <C, 0, 0, 0, 1, 0, 3> //
5 <A, 0, 0, 1, 0, 0, 3>, <B, 0, 0, 1, 0, 0, 3>, <C, 0, 0, 0, 1, 0, 3> // Accordo
```

Nel caso sfortunato in cui invece esca *Croce*, e' necessario eseguire un *round* aggiuntivo per raggiungere il consenso.

I messaggi di **D** potrebbero cambiare fra *round*, vediamo questo esempio con due lanci di moneta diversi, che superano il numero atteso di round (2):

Analizziamo il primo caso in cui l'accordo ricade sul cinema in Periferia, CROCE, TESTA:

// — **ROUND 0** — //

1 <A, 1, x, x, x, x, x>, <B, 1, x, x, x, x, x>, <C, 0, x, x, x, x, x> //

2 <A, 1, 1, 0, 0, x, x>, <B, 1, 1, 0, 1, x, x>, <C, 0, 1, 1, 1, x, x> //

3 <A, 1, 1, 0, 0, 0, x>, <B, 1, 1, 0, 1, 1, x>, <C, 0, 1, 1, 1, 1, x> //

4 <A, 1, 1, 0, 0, 0, 2>, <B, 1, 1, 0, 1, 1, 3>, <C, 0, 1, 1, 1, 1, 3> //

5 <A, 0, 1, 0, 0, 0, 2>, <B, 1, 1, 0, 1, 1, 3>, <C, 1, 1, 1, 1, 1, 3> // CROCE

// — **ROUND 1** — //

1 <A, 0, x, x, x, x, x>, <B, 1, x, x, x, x, x>, <C, 1, x, x, x, x, x> //

2 <A, 0, 1, 1, 0, x, x>, <B, 1, 0, 1, 0, x, x>, <C, 1, 0, 1, 0, x, x> //

3 <A, 0, 1, 1, 0, 0, x>, <B, 1, 0, 1, 0, 0, x>, <C, 1, 0, 1, 0, 0, x> //

4 <A, 0, 1, 1, 0, 0, 2>, <B, 1, 0, 1, 0, 0, 2>, <C, 1, 0, 1, 0, 0, 2> //

5 <A, 1, 1, 1, 0, 0, 2>, <B, 1, 0, 1, 0, 0, 2>, <C, 1, 0, 1, 0, 0, 2> // TESTA - SYNC

// — **ROUND 2** — //

1 <A, 1, x, x, x, x, x>, <B, 1, x, x, x, x, x>, <C, 1, x, x, x, x, x> //

2 <A, 1, 1, 1, 1, x, x>, <B, 1, 1, 1, 0, x, x>, <C, 1, 1, 1, 1, x, x> //

3 <A, 1, 1, 1, 1, 1, x>, <B, 1, 1, 1, 0, 1, x>, <C, 1, 1, 1, 1, 1, x> //

4 <A, 1, 1, 1, 1, 1, 4>, <B, 1, 1, 1, 0, 1, 3>, <C, 1, 1, 1, 1, 1, 4> //

5 <A, 1, 1, 1, 1, 1, 4>, <B, 1, 1, 1, 0, 1, 3>, <C, 1, 1, 1, 1, 1, 4> // **Accordo**

Analizziamo il primo caso in cui l'accordo ricade sul cinema in Centro, CROCE, CROCE:

// — **ROUND 0** — //

1 <A, 1, x, x, x, x, x>, <B, 1, x, x, x, x, x>, <C, 0, x, x, x, x, x> //

2 <A, 1, 1, 0, 0, x, x>, <B, 1, 1, 0, 1, x, x>, <C, 0, 1, 1, 1, x, x> //

3 <A, 1, 1, 0, 0, 0, x>, <B, 1, 1, 0, 1, 1, x>, <C, 0, 1, 1, 1, 1, x> //

4 <A, 1, 1, 0, 0, 0, 2>, <B, 1, 1, 0, 1, 1, 3>, <C, 0, 1, 1, 1, 1, 3> //

5 <A, 0, 1, 0, 0, 0, 2>, <B, 1, 1, 0, 1, 1, 3>, <C, 1, 1, 1, 1, 1, 3> // CROCE

// — **ROUND 1** — //

1 <A, 0, x, x, x, x, x>, <B, 1, x, x, x, x, x>, <C, 1, x, x, x, x, x> //

2 <A, 0, 1, 1, 0, x, x>, <B, 1, 0, 1, 0, x, x>, <C, 1, 0, 1, 0, x, x> //

3 <A, 0, 1, 1, 0, 0, x>, <B, 1, 0, 1, 0, 0, x>, <C, 1, 0, 1, 0, 0, x> //

4 <A, 0, 1, 1, 0, 0, 2>, <B, 1, 0, 1, 0, 0, 2>, <C, 1, 0, 1, 0, 0, 2> //

5 <A, 0, 1, 1, 0, 0, 2>, <B, 0, 0, 1, 0, 0, 2>, <C, 0, 0, 1, 0, 0, 2> // CROCE - SYNC

// — **ROUND 2** — //

1 <A, 0, x, x, x, x, x>, <B, 0, x, x, x, x, x>, <C, 0, x, x, x, x, x> //

2 <A, 0, 0, 0, 1, x, x>, <B, 0, 0, 0, 0, x, x>, <C, 0, 0, 0, 1, x, x> //

3 <A, 0, 0, 0, 1, 0, x>, <B, 0, 0, 0, 0, 0, x>, <C, 0, 0, 0, 1, 0, x> //

4 <A, 0, 0, 0, 1, 0, 3>, <B, 0, 0, 0, 0, 0, 4>, <C, 0, 0, 0, 1, 0, 3> //

5 <A, 0, 0, 0, 1, 0, 3>, <B, 0, 0, 0, 0, 0, 4>, <C, 0, 0, 0, 1, 0, 3> // **Accordo**

Possiamo notare come alla fine, qualsiasi siano le operazioni svolte da **D**, e il risultato della moneta, prima o poi si raggiungera' il consenso.

Baldini Filippo - 6393212